

UNIVERSIDAD CONTINENTAL

Pruebas y Calidad de Software



INFORME TÉCNICO DE PRUEBAS DEL PROYECTO RENTAAUTOS

Facultad: Ingeniería

EAP: Ingeniería de Sistemas e Informática

Docente: Ing. Jisbaj Gamarra Salas

Integrantes:

Ccahua Laurel Guillermo

Eddu Yerardo Panti Mozo

Laura Paz Mariano Jhoshua

NRC: 35226

Lugar: Cusco, Perú

Fecha: 06/10/2026

Presentación del informe

El presente informe documenta la estrategia, planificación, diseño, implementación y resultados de las pruebas aplicadas al proyecto RENTAAUTOS para la evaluación parcial del curso Pruebas y Calidad de Software. La organización responde a los cuatro apartados centrales solicitados en la consigna: contexto y alcance; estrategia y planificación; diseño e implementación; y resultados, análisis y conclusiones.

La evidencia se sustenta en los cuatro archivos PHP de pruebas entregados, las capturas de ejecución proporcionadas y la información técnica confirmada por el equipo. No se atribuyen funcionalidades no verificadas ni se reportan porcentajes de cobertura no medidos.

Contenido

Presentación del informe.....	2
Contenido.....	2
1. Contexto del proyecto y alcance de pruebas.....	4
1.1 Propósito y estado actual del proyecto	4
1.2 Alcance de esta evaluación.....	5
1.3 Entorno confirmado de pruebas	6
2. Estrategia y planificación de pruebas	6
2.1 Estrategia basada en riesgos	6
2.2 Tipos, niveles y dependencias	7
2.3 Orden de ejecución y criterios de aceptación	7
2.4 Planificación y coordinación	7
3. Diseño e implementación de pruebas.....	8
3.1 Inventario de pruebas propias	8
3.2 Matriz de las 15 pruebas implementadas	8
3.3 Prueba de modelo representativa	9
3.4 Prueba unitaria aislada del cálculo.....	9
3.5 Defecto detectado en la validación de disponibilidad	10
4. Resultados, análisis y conclusiones	11
4.1 Evolución de la ejecución	11
4.2 Resultado de la prueba unitaria aislada	12
4.3 Resultado final de la suite	12
4.4 Análisis técnico	13
4.5 Conclusiones	13
4.6 Mejoras pendientes	13
Anexos.....	14
Anexo A. Repositorio	14
Anexo B. Correspondencia entre evidencia y afirmación	14

1. Contexto del proyecto y alcance de pruebas

1.1 Propósito y estado actual del proyecto

RENTAAUTOS es un proyecto de alquiler de vehículos desarrollado con Laravel de la serie 12. Su núcleo de dominio incluye los modelos Auto y Reserva, así como una capa de servicio denominada ReservaService. Según la información técnica proporcionada por el equipo, composer.json declara laravel/framework ^12.0; por ello, se identifica el proyecto dentro de la serie Laravel 12 sin afirmar una versión exacta instalada.

El modelo Auto se relaciona con múltiples reservas mediante una relación hasMany, mientras que Reserva pertenece a un Auto mediante belongsTo y realiza casts de sus campos de fecha. Las migraciones establecen estados por defecto y restricciones de persistencia. En la capa de aplicación, el formulario permite mostrar vehículos, comprobar disponibilidad y calcular un importe, pero no persiste una reserva. La operación crearReserva del servicio sí crea la reserva y concentra validaciones de estado del vehículo, disponibilidad temporal y cálculo del total.

El sistema se encuentra en desarrollo. Este informe no lo presenta como producto terminado y no atribuye al proyecto pagos, notificaciones, autenticación ni un flujo HTTP completo de creación de reservas verificado.

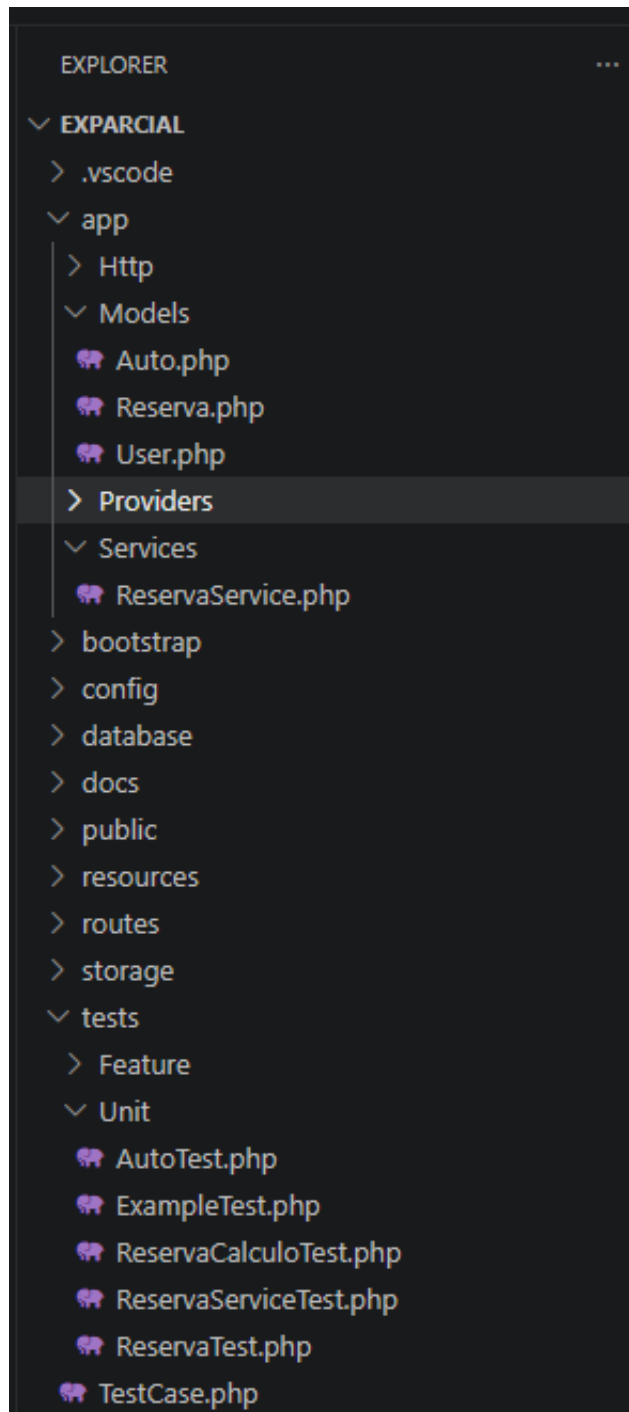


Figura 1. Estructura actual relevante del proyecto RENTAAUTOS.

La Figura 1 ubica los elementos relevantes para esta entrega: modelos Auto y Reserva, el servicio ReservaService y los archivos de prueba AutoTest, ReservaTest, ReservaServiceTest y ReservaCalculoTest. La ubicación de un archivo bajo tests/Unit no determina por sí sola que todas sus pruebas sean unitarias aisladas; la clasificación depende de sus dependencias y del uso de persistencia.

1.2 Alcance de esta evaluación

El alcance se centra en comportamientos de dominio que presentan riesgo funcional directo para el alquiler de vehículos: persistencia básica de autos y reservas, configuración de atributos, relaciones

entre modelos, estados por defecto, casteo de fechas, cálculo del importe, validación de fechas y detección de reservas superpuestas. También se verifica que crearReserva rechace un auto en mantenimiento; no se incluye una prueba de creación exitosa.

Dentro del alcance	Fuera del alcance de esta entrega
Modelos Auto y Reserva, relaciones y persistencia	Pruebas completas de interfaz gráfica
Estados por defecto y casts de fechas	Rendimiento y carga
ReservaService: disponibilidad, fechas, creación y total	Seguridad y pruebas de penetración
Prueba unitaria aislada del cálculo del importe	Concurrencia y condiciones de carrera
Defecto documentado de solapamiento y su corrección	Integraciones externas
Ejecución automatizada mediante php artisan test	Cobertura porcentual, porque no se midió en la evidencia entregada

1.3 Entorno confirmado de pruebas

- Sistema operativo: Windows.
- Editor y terminal: VS Code y PowerShell.
- PHP 8.4.24.
- Composer 2.10.3.
- PHPUnit 11.5.56.
- Ejecución mediante php artisan test.
- phpunit.xml: APP_ENV=testing, DB_CONNECTION=sqlite y DB_DATABASE=:memory:.
- Los casos con RefreshDatabase utilizan una base de datos de pruebas y las migraciones, sin depender de registros manuales previos.

2. Estrategia y planificación de pruebas

2.1 Estrategia basada en riesgos

La estrategia prioriza los riesgos que pueden producir una reserva inválida o un cobro incorrecto. La mayor criticidad se asigna a la detección de periodos superpuestos, porque permitir dos reservas activas para el mismo vehículo compromete la regla central de disponibilidad. A continuación se priorizan las fechas incoherentes, el cálculo del importe, las relaciones Auto-Reserva y los estados por defecto que sostienen la persistencia esperada.

Riesgo	Impacto	Prioridad	Respuesta de prueba
Periodos superpuestos	Doble asignación de un vehículo	Alta	Casos de conflicto temporal en ReservaServiceTest
Fechas incoherentes	Reserva con intervalo inválido	Alta	Excepción para fecha_fin no posterior
Importe incorrecto	Cobro errado	Alta	Cálculo integrado y prueba unitaria aislada
Relaciones incorrectas	Inconsistencia entre auto y reserva	Media-Alta	hasMany y belongsTo
Estados por defecto/persistencia	Datos en estado no esperado	Media-Alta	Persistencia y valores por defecto

2.2 Tipos, niveles y dependencias

La clasificación se realiza por el comportamiento y las dependencias de cada prueba, no por la carpeta donde está almacenado el archivo:

- **AutoTest y ReservaTest:** pruebas de modelo; los casos que crean registros integran Eloquent, migraciones y la base SQLite en memoria mediante RefreshDatabase.
- **ReservaServiceTest:** pruebas del servicio con integración a persistencia, porque usa modelos Eloquent, RefreshDatabase y consultas sobre reservas.
- **ReservaCalculoTest:** prueba unitaria aislada de la base de datos: hereda de PHPUnit\Framework\TestCase, instancia Auto en memoria y no usa RefreshDatabase.
- **ExampleTest unitario y Feature:** ejemplos de Laravel presentes en la suite total, pero no se usan como evidencia principal del negocio.

2.3 Orden de ejecución y criterios de aceptación

El orden adoptado comienza por pruebas de modelos y reglas básicas de persistencia, continúa con el servicio de reservas y termina con la suite completa. Cuando se incorpora un caso que revela un defecto, se ejecuta primero de forma filtrada para reproducirlo, luego se corrige la condición y se repite el mismo caso antes de volver a ejecutar la suite. Esta secuencia reduce el ruido y permite relacionar el cambio con el comportamiento observado.

Un caso se considera aceptado cuando finaliza sin fallos y cumple sus aserciones; en pruebas negativas también debe producirse la clase de excepción y, cuando se especifica, el fragmento de mensaje esperado.. La suite debe ser reproducible en el entorno de pruebas configurado. La aprobación de la suite no implica cobertura total de escenarios ni ausencia absoluta de defectos.

2.4 Planificación y coordinación

Actividad	Prioridad	Orden	Coordinación	Estado	Evidencia
Verificar modelos Auto y Reserva	Alta	1	Ccahua Laurel Guillermo	Realizado	AutoTest y ReservaTest
Validar reglas del servicio	Alta	2	Eddu Yerardo Panti Mozo	Realizado	ReservaServiceTest
Reproducir y corregir solapamiento	Alta	3	Eddu Yerardo Panti Mozo	Realizado	Fallo y reejecución del caso filtrado
Aislar cálculo del importe	Alta	4	Laura Paz Mariano Jhoshua	Realizado	ReservaCalculoTest
Ejecutar suite final y consolidar resultados	Alta	5	Equipo	Realizado	17 pruebas, 31 aserciones
Ampliar casos límite y medir cobertura	Media	Futuro	Equipo	Pendiente	Trabajo de siguiente iteración

El reparto anterior se presenta como coordinación para la sustentación y no como autoría exclusiva ni como reconstrucción histórica de todas las tareas. El equipo participa conjuntamente en implementación, ejecución, análisis y revisión.

3. Diseño e implementación de pruebas

3.1 Inventario de pruebas propias

La suite final contiene 17 pruebas: 15 corresponden directamente al proyecto y 2 son ExampleTest incluidos por Laravel. Las 15 pruebas propias se distribuyen en AutoTest (4), ReservaTest (4), ReservaServiceTest (6) y ReservaCalculoTest (1).

3.2 Matriz de las 15 pruebas implementadas

ID	Componente	Tipo	Comportamiento / riesgo	Datos de entrada	Resultado esperado	Resultado final
AUT-01	AutoTest	Modelo + BD	Crear un auto	TEST-001, Toyota Corolla 2023, 150, disponible	Registro existente y placa TEST-001	Aprobado
AUT-02	AutoTest	Modelo (metadatos)	Atributos fillable	Instancia Auto	Incluye placa, marca, modelo, año, tarifa_diaria y estado	Aprobado
AUT-03	AutoTest	Modelo + BD	Relación Auto hasMany Reserva	Auto TEST-002 + una reserva	Una reserva asociada de tipo Reserva	Aprobado
AUT-04	AutoTest	Modelo + BD	Estado por defecto del auto	Auto TEST-003 sin estado explícito	estado = disponible	Aprobado
RES-01	ReservaTest	Modelo + BD	Crear una reserva	Auto TEST-100 + Juan Pérez, 1-5 nov, total 560	Registro existente y total 560	Aprobado
RES-02	ReservaTest	Modelo + BD	Casts de fechas	Reserva 1-5 dic 2026	fecha_inicio y fecha_fin son Carbon	Aprobado
RES-03	ReservaTest	Modelo + BD	Relación Reserva belongsTo Auto	Auto TEST-102 + reserva	Reserva devuelve Auto y placa TEST-102	Aprobado
RES-04	ReservaTest	Modelo + BD	Estado por defecto de reserva	Reserva sin estado explícito	estado = activa	Aprobado
SRV-01	ReservaServiceTest	Servicio + BD	Contención completa de periodos	Existente 10-20 nov; solicitud 12-15 nov	InvalidArgumentException: auto no disponible	Aprobado tras corrección
SRV-02	ReservaServiceTest	Servicio + BD	Solapamiento parcial	Existente 15-20 oct; solicitud 18-22 oct	InvalidArgumentException: auto no disponible	Aprobado
SRV-03	ReservaServiceTest	Servicio + BD	Vehículo disponible	Auto sin reserva; 1-5 nov	validarDisponibilidad retorna true	Aprobado
SRV-04	ReservaServiceTest	Servicio + BD	Cálculo de total	Tarifa 120; 1-5 nov	total = 480.00	Aprobado
SRV-05	ReservaServiceTest	Servicio	Fechas inválidas	Inicio 10 nov; fin 5 nov	InvalidArgumentException: fecha fin posterior	Aprobado
SRV-06	ReservaServiceTest	Servicio + BD	No crear reserva si auto no disponible	Auto en mantenimiento; 1-5 dic	InvalidArgumentException: auto no disponible	Aprobado
UNI-01	ReservaCalculoTest	Unitaria aislada	Cálculo sin base de datos	Auto en memoria, tarifa 120; 1-5 nov	float 480.00 con assertSame	Aprobado

3.3 Prueba de modelo representativa

AutoTest utiliza Tests\TestCase y RefreshDatabase. El caso test_un_auto_se_puede_crear prepara un registro de Auto mediante Eloquent, ejecuta la creación y verifica tanto la persistencia en la tabla autos como el valor de la placa del objeto resultante.

```
<?php
public function test_un_auto_se_puede_crear(): void
{
    $auto = Auto::create([
        'placa' => 'TEST-001',
        'marca' => 'Toyota',
        'modelo' => 'Corolla',
        'anio' => 2023,
        'tarifa_diaria' => 150.00,
        'estado' => 'disponible',
    ]);

    $this->assertDatabaseHas('autos', [
        'placa' => 'TEST-001',
        'marca' => 'Toyota',
    ]);

    $this->assertEquals('TEST-001', $auto->placa);
}
```

Figura 2. Prueba de modelo test_un_auto_se_puede_crear en AutoTest.php.

```
$auto = Auto::create([
    'placa' => 'TEST-001',
    'marca' => 'Toyota',
    'modelo' => 'Corolla',
    'anio' => 2023,
    'tarifa_diaria' => 150.00,
    'estado' => 'disponible',
]);
$this->assertDatabaseHas('autos', ['placa' => 'TEST-001', 'marca' => 'Toyota']);
$this->assertEquals('TEST-001', $auto->placa);
```

Preparación: se definen los datos válidos del vehículo. Acción: Auto::create persiste el modelo. Aserciones: assertDatabaseHas confirma la presencia del registro y assertEquals valida la placa del objeto creado. Por depender de Eloquent y de una base de pruebas, este caso se clasifica como prueba de modelo con integración a persistencia, no como prueba unitaria aislada.

3.4 Prueba unitaria aislada del cálculo

ReservaCalculoTest fue incorporada para aislar el cálculo del importe respecto de la base de datos. El archivo hereda directamente de PHPUnit\Framework\TestCase, crea un Auto en memoria, asigna tarifa_diaria y llama al método calcularTotal. No usa RefreshDatabase ni guarda el modelo.

```

1  <?php
2
3  namespace Tests\Unit;
4
5  use App\Models\Auto;
6  use App\Services\ReservaService;
7  use PHPUnit\Framework\TestCase;
8
9  class ReservaCalculoTest extends TestCase
10 {
11     public function test_calcula_importe_sin_base_de_datos(): void
12     {
13         // Preparación: objeto en memoria, sin persistencia.
14         $auto = new Auto();
15         $auto->tarifa_diaria = 120.00;
16         $service = new ReservaService();
17
18         // Acción: cuatro días de alquiler.
19         $total = $service->calcularTotal($auto, '2026-11-01', '2026-11-05');
20
21         // Verificación: cuatro días por una tarifa de 120.
22         $this->assertSame(480.00, $total);
23     }
24 }
25

```

Figura 3. Código de ReservaCalculoTest con preparación, acción y verificación.

```

$auto = new Auto();
$auto->tarifa_diaria = 120.00;
$service = new ReservaService();
$total = $service->calcularTotal($auto, '2026-11-01', '2026-11-05');
$this->assertSame(480.00, $total);

```

Preparación: objeto Auto solo en memoria con tarifa 120.00. Acción: se calcula el total para cuatro días. Aserción: assertSame exige que el resultado sea exactamente el float 480.00. Esta separación reduce dependencias y permite identificar fallos del cálculo sin involucrar persistencia.

3.5 Defecto detectado en la validación de disponibilidad

La suite inicial aprobaba sus 15 pruebas, pero no incluía el escenario en el que el periodo solicitado queda completamente dentro de una reserva activa existente. Se incorporó el caso con reserva activa del 10 al 20 de noviembre de 2026 y solicitud del 12 al 15. El caso esperaba InvalidArgumentException por no disponibilidad y inicialmente falló, lo que evidenció una brecha real en la condición de solapamiento.

La consulta anterior comprobaba si el inicio o el fin de la reserva existente se encontraban dentro del periodo solicitado. Esa lógica no detectaba el caso inverso: una solicitud completamente contenida en la reserva ya existente.

```

<?php
public function validarDisponibilidad(Auto $auto, string $fechaInicio, string $fechaFin): bool
{
    if ($fechaInicio >= $fechaFin) {
        throw new InvalidArgumentException('La fecha de fin debe ser posterior a la fecha de inicio.');
```

```

    }

    $conflicto = Reserva::where('auto_id', $auto->id)
        ->where('estado', 'activa')
        ->where('fecha_inicio', '<=', $fechaFin)
        ->where('fecha_fin', '>=', $fechaInicio)
        ->exists();

    if ($conflicto) {
        throw new InvalidArgumentException('El auto no está disponible en las fechas seleccionadas.');
```

```

    }

    return true;
}

```

Figura 4. Condición corregida para detectar intervalos superpuestos en ReservaService.

```

->where('fecha_inicio', '<=', $fechaFin)
->where('fecha_fin', '>=', $fechaInicio)

```

La corrección considera conflicto cuando el inicio de una reserva existente es anterior o igual al fin solicitado y, simultáneamente, su fin es posterior o igual al inicio solicitado. Con estas dos condiciones se detecta la intersección de intervalos, incluida la contención completa. La implementación también considera conflicto cuando los intervalos comparten una fecha extrema. Este comportamiento se documenta, aunque no se afirma que todos los límites posibles hayan sido probados. Tampoco se atribuye a esta consulta la resolución de reservas concurrentes.

4. Resultados, análisis y conclusiones

4.1 Evolución de la ejecución

La evidencia muestra una evolución controlada de la suite. Primero se ejecutaron las pruebas existentes; después se añadió un caso de solapamiento que falló, se corrigió la consulta, se verificó el mismo caso y posteriormente se incorporó una prueba unitaria aislada del cálculo. Finalmente se ejecutó nuevamente toda la suite.

```
PS C:\Users\yabar\Desktop\exParcial> php artisan test

PASS Tests\Unit\AutoTest
✓ un auto se puede crear 0.00s
✓ un auto tiene atributos fillable 0.00s
✓ un auto tiene muchas reservas 0.00s
✓ un auto por defecto es disponible 0.00s

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Unit\ReservaServiceTest
✓ no permite reservar auto no disponible 0.00s
✓ permite reservar auto disponible 0.00s
✓ calcula total correctamente 0.00s
✓ no permite fechas invalidas 0.00s
✓ no crea reserva si auto no esta disponible 0.00s

PASS Tests\Unit\ReservaTest
✓ una reserva se puede crear 0.00s
✓ una reserva crea fechas correctamente 0.00s
✓ una reserva pertenece a un auto 0.00s
✓ una reserva por defecto es activa 0.00s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.10s

Tests: 15 passed (28 assertions)
Duration: 1.16s

PS C:\Users\yabar\Desktop\exParcial>
```

Figura 5. Suite inicial: 15 pruebas aprobadas, 28 aserciones y 1.16 s.

La Figura 5 confirma que la suite disponible en ese momento finalizó sin fallos. Sin embargo, este resultado no garantizaba que la regla de solapamiento estuviera completamente cubierta; el escenario de contención aún no formaba parte de los casos automatizados.

```
PS C:\Users\yabar\Desktop\exParcial> php artisan test --filter=test_rechaza_reserva_dentro_de_otra_existente

FAIL Tests\Unit\ReservaServiceTest
✗ rechaza reserva dentro de otra existente

FAILED Tests\Unit\ReservaServiceTest > rechaza reserva dentro de otra existente
Failed asserting that exception of type "InvalidArgumentException" is thrown.

Tests: 1 failed (1 assertions)
Duration: 0.51s
```

Figura 6. Fallo del nuevo caso de contención de periodos: 1 prueba fallida, 1 aserción, 0.51 s.

La ejecución filtrada del caso test_rechaza_reserva_dentro_de_otra_existente falló porque no se lanzó la InvalidArgumentException esperada. Técnicamente, esto demostró que la lógica previa aceptaba un intervalo solicitado totalmente contenido dentro de una reserva activa.

```
PS C:\Users\yabar\Desktop\exParcial> php artisan test --filter=test_rechaza_reserva_dentro_de_otra_existente

PASS Tests\Unit\ReservaServiceTest
✓ rechaza reserva dentro de otra existente

Tests: 1 passed (2 assertions)
Duration: 0.50s
```

Figura 7. Reejecución del caso de solapamiento después de la corrección: 1 prueba aprobada, 2 aserciones y 0.50 s.

Después de ajustar la consulta, el mismo caso pasó. La reejecución sobre el mismo escenario permite vincular la corrección con el comportamiento esperado y deja una prueba de regresión para evitar que esta brecha reaparezca sin ser detectada.

4.2 Resultado de la prueba unitaria aislada

```
PS C:\Users\yabar\Desktop\exParcial> php artisan test --filter=ReservaCalculoTest

PASS Tests\Unit\ReservaCalculoTest
✓ calcula importe sin base de datos

Tests: 1 passed (1 assertions)
Duration: 0.19s
```

Figura 8. Ejecución de ReservaCalculoTest: 1 prueba aprobada, 1 aserción y 0.19 s.

La Figura 8 confirma el comportamiento del cálculo sin persistencia: una tarifa diaria de 120.00 aplicada a cuatro días produce 480.00. Esta prueba complementa al caso integrado del servicio porque reduce el conjunto de dependencias implicadas en el resultado.

4.3 Resultado final de la suite

```
PS C:\Users\yabar\Desktop\exParcial> php artisan test

PASS Tests\Unit\AutoTest
✓ un auto se puede crear
✓ un auto tiene atributos fillable
✓ un auto tiene muchas reservas
✓ un auto por defecto es disponible

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Unit\ReservaCalculoTest
✓ calcula importe sin base de datos

PASS Tests\Unit\ReservaServiceTest
✓ rechaza reserva dentro de otra existente
✓ no permite reservar auto no disponible
✓ permite reservar auto disponible
✓ calcula total correctamente
✓ no permite fechas invalidas
✓ no crea reserva si auto no esta disponible

PASS Tests\Unit\ReservaTest
✓ una reserva se puede crear
✓ una reserva castea fechas correctamente
✓ una reserva pertenece a un auto
✓ una reserva por defecto es activa

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response

Tests: 17 passed (31 assertions)
Duration: 0.84s
```

Figura 9. Suite final de RENTAAUTOS: 17 pruebas aprobadas, 31 aserciones y 0.84 s.

La ejecución final integra los casos de modelos, servicio, prueba unitaria aislada y los dos ExampleTest de Laravel. La suite final reporta 17 pruebas aprobadas y 31 aserciones en 0.84 s. No se suman tiempos ni aserciones de ejecuciones filtradas independientes para construir este total.

Grupo	Casos	Estado final	Observación
AutoTest	4	Aprobados	Persistencia, fillable, relación y estado por defecto
ReservaTest	4	Aprobados	Persistencia, casts, relación y estado por defecto
ReservaServiceTest	6	Aprobados	Disponibilidad, fechas, total,

Grupo	Casos	Estado final	Observación
			creación y solapamiento
ReservaCalculoTest	1	Aprobado	Cálculo aislado sin base de datos
ExampleTest Laravel	2	Aprobados	Ejemplos incluidos en el total; no son evidencia principal del dominio
Suite completa	17	Aprobada	31 aserciones; 0.84 s

4.4 Análisis técnico

El resultado más relevante no es únicamente que la suite final esté aprobada, sino que una suite inicialmente verde pudo omitir un defecto de negocio porque no contenía un caso representativo de contención temporal. La adición de ese escenario transformó un riesgo teórico en un fallo reproducible, guio una corrección concreta y quedó incorporada como protección frente a regresiones.

Las pruebas de modelo aportan confianza sobre la interacción de Eloquent con la base de datos de pruebas, especialmente relaciones, persistencia, casts y valores por defecto. La prueba unitaria aislada aporta una señal más específica sobre el cálculo al eliminar la dependencia de la persistencia. Ambos enfoques son complementarios y justifican que la clasificación no se base únicamente en la carpeta tests/Unit.

Los resultados no permiten afirmar cobertura completa ni ausencia total de defectos. Tampoco se ha medido un porcentaje de cobertura. Permanecen fuera de lo comprobado los casos completos de interfaz, seguridad, rendimiento, concurrencia e integraciones externas.

4.5 Conclusiones

1. La estrategia basada en riesgos permitió concentrar las pruebas en reglas críticas del alquiler: disponibilidad temporal, coherencia de fechas, cálculo del importe, relaciones de dominio y persistencia.
2. AutoTest y ReservaTest validan el comportamiento de los modelos con base de datos de pruebas, mientras que ReservaCalculoTest demuestra un caso unitario aislado del mecanismo de persistencia.
3. La suite inicial aprobada no era suficiente para demostrar que todos los escenarios de solapamiento estuvieran cubiertos. El nuevo caso de contención expuso una brecha específica que no había sido detectada por los casos existentes.
4. La corrección de la consulta y la aprobación posterior del mismo caso proporcionan evidencia directa de que el escenario documentado quedó cubierto y ayudan a prevenir su regresión.
5. La suite final aprobó 17 pruebas con 31 aserciones. Este resultado demuestra consistencia de los casos ejecutados, pero no equivale a cobertura total del sistema.

4.6 Mejoras pendientes

- Agregar casos explícitos de fechas adyacentes y límites temporales para documentar con precisión el criterio de conflicto.
- Incorporar escenarios de reservas canceladas y su efecto en la disponibilidad.
- Agregar validaciones HTTP cuando el flujo de creación de reservas a nivel de interfaz/controlador esté definido y sea parte del alcance.
- Diseñar pruebas de concurrencia para estudiar solicitudes simultáneas sobre el mismo vehículo.
- Medir cobertura con una herramienta adecuada y reportarla solo cuando exista evidencia reproducible.

- La principal dificultad fue identificar un escenario de solapamiento ausente en las pruebas iniciales. También fue necesario separar el cálculo de la persistencia para demostrar una prueba unitaria aislada.

Anexos

Anexo A. Repositorio

Repositorio: <https://github.com/AXXELYSS/rentaautos-sistema>

El repositorio contiene el código y las pruebas implementadas

Anexo B. Correspondencia entre evidencia y afirmación

Figura	Evidencia	Afirmación respaldada
1	Estructura actual	Existencia y ubicación visual de modelos, servicio y archivos de prueba
2	Código AutoTest	Ejemplo de prueba de modelo con persistencia
3	Código ReservaCalculoTest	Prueba unitaria aislada, sin RefreshDatabase
4	Consulta corregida	Condición de intersección temporal implementada
5	Suite inicial	15 pruebas, 28 aserciones, 1.16 s
6	Caso fallido	Defecto reproducido en contención completa
7	Caso corregido	Mismo escenario aprobado tras la corrección
8	Prueba unitaria	Cálculo aislado aprobado
9	Suite final	17 pruebas, 31 aserciones, 0.84 s